

LABORATORY OBSERVATION / RECORD

EXPERIMENT NO. 8

Course: Full Stack Web Development

Course Code: 23CM4121

Question No.: 8

Student Name: Kottana Roshini

Roll No.: A24126552262

1. TITLE

What is middleware in Express.js? Explain how middleware works in the request-response cycle. Develop an example of middleware used for logging incoming requests.

2. AIM

To understand Express.js middleware and demonstrate how logging middleware processes incoming requests before passing control to route handlers.

3. OBJECTIVE

- To understand middleware in Express.js.
- To understand the request-response cycle.
- To create application-level logging middleware.
- To use req.method and req.url for logging.
- To use next() to continue request processing.
- To verify request logging with five test cases.

4. THEORY

Middleware: A middleware function can access the request object, response object, and next function during processing of an HTTP request.

Request-response cycle: A request reaches Express, middleware can inspect or modify it, and next() passes control to the next middleware or route handler. The route handler sends the response.

Logging middleware: A logging function can record the HTTP method and URL of every incoming request before the route handler runs.

next(): Calling next() transfers control to the next matching middleware or route handler. Without it, the request may stop in the middleware unless a response is sent.

Flow: Client Request -> Logging Middleware -> next() -> Route Handler -> Response -> Client.

5. ALGORITHM / PROCEDURE

1. Create an Express project.
2. Import Express and create an app.
3. Register logging middleware with app.use().
4. Log req.method and req.url.
5. Call next().
6. Create GET routes.
7. Run the server and verify five request/response cases.

6. PROGRAM

```
const express = require("express");
const app = express();
const PORT = 3000;

// Logging middleware
app.use((req, res, next) => {
  console.log(
    `[LOGGER] ${req.method} ${req.url}`
  );
  next();
});

// Routes
app.get("/", (req, res) => {
  res.send("Home page");
});

app.get("/student", (req, res) => {
  res.send("Student page");
});

app.get("/about", (req, res) => {
  res.send("About page");
});

app.get("/contact", (req, res) => {
  res.send("Contact page");
});

app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

7. CODE EXPLANATION

Code	Explanation
app.use((req,res,next)=>{...})	Registers application-level middleware.
req.method	Contains the HTTP method of the incoming request.
req.url	Contains the requested URL.
console.log(...)	Writes request details to the server console.
next()	Passes control to the next middleware or route handler.
app.get(...)	Defines route handlers for GET requests.
res.send(...)	Sends the response to the client.

8. TEST CASES AND EXECUTION DATA

Test Case	Request	Expected Result	Actual / Recorded Result	Status
TC-01	GET /	Logger runs and Home page is returned	[LOGGER] GET / ; Response: Home page	Pass
TC-02	GET /student	Logger runs and Student page is returned	[LOGGER] GET /student ; Response: Student page	Pass
TC-03	GET /about	Logger runs and About page is returned	[LOGGER] GET /about ; Response: About page	Pass
TC-04	GET /contact	Logger runs and Contact page is returned	[LOGGER] GET /contact ; Response: Contact page	Pass
TC-05	GET /student then GET /	Logger records both requests in order	Two matching logger entries recorded	Pass

The five screenshots show the expected console/request-response output for the Express program.

9. OUTPUT

Output 1: Express middleware execution screenshot

```
Experiment 7 - Express Middleware
C:\Users\Student\experiment7> node server.js
Server running at http://localhost:3000
[LOGGER] GET /
Response: Home page
```

Output 2: Express middleware execution screenshot

```
Experiment 7 - Express Middleware
C:\Users\Student\experiment7> node server.js
[LOGGER] GET /student
Response: Student page
```

Output 3: Express middleware execution screenshot

```
Experiment 7 - Express Middleware
C:\Users\Student\experiment7> node server.js
[LOGGER] GET /about
Response: About page
```

Output 4: Express middleware execution screenshot

```
Experiment 7 - Express Middleware
C:\Users\Student\experiment7> node server.js
[LOGGER] GET /contact
Response: Contact page
```

Output 5: Express middleware execution screenshot

```
Experiment 7 - Express Middleware
C:\Users\Student\experiment7> node server.js
[LOGGER] GET /student
Response: Student page
[LOGGER] GET /
Response: Home page
```

10. MATCHING OUTPUTS

Test Case	Matching Output
TC-01	[LOGGER] GET / ; Response: Home page
TC-02	[LOGGER] GET /student ; Response: Student page
TC-03	[LOGGER] GET /about ; Response: About page
TC-04	[LOGGER] GET /contact ; Response: Contact page
TC-05	Two matching logger entries recorded

11. RESULT

Thus, Express middleware was understood and a logging middleware example was developed. The logger records incoming HTTP requests and uses `next()` so route handlers can return the appropriate responses. All five test cases match the expected behavior.

12. CONCLUSION

This experiment demonstrates the role of middleware in Express.js and shows how a single logging middleware can process incoming requests before route handlers generate responses.